카드 및 전술스킬 이펙트 UI 연출 분리 아키텍처 보고서

Unified Visual Cue Event Bus Architecture & Git Collaborative Decoupling

프로젝트: WildCard (Unreal Engine 5.7) **문서 번호:** WC-REP-2026-10-ARCH-01 **작성일:** 2026-10-14

작성자: Core Architecture Team **빌드 검증:** WildCardEditor Win64 Development (Succeeded)

1. Executive Summary (개요 및 요약)

본 보고서는 WildCard 프로젝트의 카드 효과(Card Effect)와 영웅 전술스킬(Tactical Skill) 발동 시 발생하는 **시각적 연출(UI 애니메이션, VFX, SFX)** 파이프라인의 구조적 결합도를 해소하고, **Git 기반 다중 작업자 환경에서 언리얼 바이너리 에셋(WBP_HUD)의 충돌을 원천 차단**하기 위해 수행된 통합 아키텍처 설계 및 C++ 리팩터링 결과를 종합 정리합니다.

핵심 성과 요약:

- **단일 NetMulticast 이벤트 버스 구축:** 개별 이펙트/스킬별 RPC 난립을 제거하고, `FWildVisualCueData` 구조체와 `AInGameState::Multicast_BroadcastVisualCue` 기반의 단일 경량 통지 채널 확립.
- **UI 연출 모듈의 완전 독립화 (Zero-Touch HUD):** 중앙 HUD 위젯(`UWildHUDWidget`)에 개별 연출 로직이 하드코딩되던 구조를 타파하고, `UWildVisualEffectWidget` 을 상속받는 독립 위젯이 계층형 GameplayTag 매핑을 통해 동적 스폰 및 자가 소멸(Self-Destruction)하도록 구현.
- **서고 탐색(Search Library) 파이프라인 정립:** '로컬 카드 선택 모달(Input Phase)'과 '서버 이펙트 실행 및 광역 UI 연출(Presentation Phase)'을 깔끔하게 분리하여 동기화 안정성 및 재사용성 극대화.
- **컴파일 및 무결성 검증 완료:** 신규/수정된 10종의 C++ 클래스 및 데이터 구조가 `WildCardEditor` 및 `WildCardServer` 타깃에서 결함 없이 빌드 성공.

2. 배경 및 기존 아키텍처의 한계 분석

2.1 전술스킬과 카드 이펙트 간 아키텍처 불균형

기존 코드베이스는 전술스킬과 카드 이펙트의 발동 메커니즘이 완전히 이원화되어 유지보수와 연출 확장에 심각한 기술 부채를 유발하고 있었습니다.

구분	기존 전술스킬 (Tactical Skill)	기존 카드 효과 (Card Effect)
실행 주체	`APlayerUnit::ExecuteTacticalSkillAction`	`UChainComponent` & `UEffectBase` 상속 클래스
연출 트리거	단일 하드코딩 프로퍼티 (`AbilityVFX` , `AbilitySound` , `OnHeroAbilityTriggered`)	**시각/UI 연출 트리거 통로 전무 (수치/상태 로직만 처리)**
문제점	스킬이 추가될 때마다 `APlayerUnit` 에 필드가 늘어나며, 스킬별 특수 연출(UI 팝업 등) 분기 불가	데미지/힐/카드 드로우 등의 효과가 발동해도 화면에 시각적 피드백을 전달할 방법 부재

2.2 '서고 탐색(Search Library)' 특수 파이프라인 분석

마법사 전술스킬인 '서고 탐색'은 일반적인 전투 스킬과 달리 **[UI 입력 대기 모달]**과 **[결과 실행 및 UI 연출]**이 복합된 구조를 가집니다.

- **입력 모달의 결합 문제:** 기존에는 `UMageSearchLibraryTacticalSkill::InitiateSkill` 내부에서 `AWildPlayerController`의 특정 래퍼 함수(`Client_ShowLibrarySearchUI`)를 직접 호출하여, 다른 클래스가 이 모달을 재사용할 수 없었습니다.
- **실행 후 시각 피드백 결여:** 플레이어가 카드를 선택하면 `USearchLibraryEffect`가 서버에서 덱을 조작하고 드로우를 수행하지만, 해당 카드가 어떤 연출로 플레이어 손패로 들어오는지 전체 클라이언트에 통지하는 파이프라인이 전무했습니다.

2.3 Git 협업 시 발생하는 치명적 바이너리 충돌 문제

> **바이너리 머지 충돌 위험 (WBP_HUD Binary Conflict):**
> 기존 구조에서는 새로운 이펙트나 스킬 연출(예: 화염구 폭발 UI, 힐링 이펙트, 서고 탐색 애니메이션 등)이 생길 때마다 모든 연출 코드가 메인 HUD 블루프린트인 `WBP_HUD` 내부의 거대한 Event Graph나 C++ 함수에 추가되어야 했습니다. 언리얼 엔진의 `.uasset` 파일은 바이너리 형식이므로 Git 머지 툴로 자동 병합이 불가능하며, 작업자 간 덮어쓰기 손실이나 개발 병목이 발생했습니다.

3. 통합 시각 효과 이벤트 버스 아키텍처

이러한 문제를 근본적으로 해결하기 위해 **GameplayTag 기반의 이벤트 버스**와 **독립 연출 위젯(UWildVisualEffectWidget)** 시스템을 구축했습니다.

3.1 핵심 아키텍처 흐름도

1. Trigger / Action
전술스킬 실행 or 체인 컴포넌트 마법 발동
`CastSpell()` / `ApplyEffect()`

2. Visual Cue 브로드캐스트
서버 권한 검증 후 `AInGameState` 의 `Multicast_BroadcastVisualCue` 호출

3. 로컬 Event Bus 수신
클라이언트 InGameState의 `OnVisualCueReceivedDelegate` 가 HUD로 데이터 전파

4. 태그 매핑 & 위젯 재생
`VisualEffectWidgetMap` 에서 계층형 매칭 후 독립 위젯 스폰
애니메이션 후 자가 소멸

3.2 FWildVisualCueData 명세

네트워크 대역폭을 최소화하면서 2D UI 연출 및 3D 공간 연출에 필요한 모든 컨텍스트를 전달할 수 있도록 통합 데이터 구조체를 정의했습니다 (`Source/WildCard/InGame/Data/BattleData.h`).

```
USTRUCT(BlueprintType)
struct FWildVisualCueData
{
    GENERATED_BODY()

    UPROPERTY(BlueprintReadWrite, Category = "VisualCue")
    FGameplayTag CueTag;                // 예: "Visual.TacticalSkill.Mage.SearchLibrary", "Visual.Effect.Dama

    UPROPERTY(BlueprintReadWrite, Category = "VisualCue")
    TWeakObjectPtr<AActor> Instigator;  // 스킬/카드 시전자

    UPROPERTY(BlueprintReadWrite, Category = "VisualCue")
    TWeakObjectPtr<AActor> Target;      // 대상 유닛 또는 타일

    UPROPERTY(BlueprintReadWrite, Category = "VisualCue")
    FVector WorldLocation = FVector::ZeroVector; // 3D 월드 투영 좌표

    UPROPERTY(BlueprintReadWrite, Category = "VisualCue")
    int32 Value = 0;                    // 데미지 수치, 힐량, 선택 카드 인덱스 등

    UPROPERTY(BlueprintReadWrite, Category = "VisualCue")
    FName ContextPayload = NAME_None;   // 추가 컨텍스트 (카드 ID, 속성 등)
};
```

3.3 AInGameState의 단일 NetMulticast 브로드캐스트

이펙트나 스킬마다 별도의 NetMulticast RPC를 추가하지 않고, `AInGameState`가 단일 통로 역할을 수행합니다.

```
// InGameState.h
UFUNCTION(NetMulticast, Unreliable)
void Multicast_BroadcastVisualCue(const FWildVisualCueData& CueData);

DECLARE_MULTICAST_DELEGATE_OneParam(FOnVisualCueReceived, const FWildVisualCueData&);
FOnVisualCueReceived OnVisualCueReceivedDelegate;

// InGameState.cpp
void AInGameState::Multicast_BroadcastVisualCue_Implementation(const FWildVisualCueData& CueData)
{
    OnVisualCueReceivedDelegate.Broadcast(CueData);
}
```

3.4 UWildVisualEffectWidget 독립 연출 위젯 시스템

기존 `WBP_HUD` 내부의 애니메이션 종속성을 분리하기 위해, 독립적으로 동작하고 스스로 수명을 관리하는 베이스 위젯을 신설했습니다.

- **`PlayVisualEffect(const FWildVisualCueData& CueData)`**: 위젯 생성 시 호출되는 BlueprintImplementableEvent로, 위젯 블루프린트에서 UMG 애니메이션이나 사운드, 파티클을 자유롭게 재생합니다.
- **`FinishVisualEffect()`**: 연출 종료 시 호출되어 뷰포트에서 즉시 제거(`RemoveFromParent`)되거나 캐싱 풀로 반환되어 메모리 누수를 방지합니다.

4. '서고 탐색' 인터랙티브 전술스킬 & UI 연출 리팩터링

사용자 요구사항에 맞추어, 가장 복잡한 파이프라인인 마법사 전술스킬 '서고 탐색'을 신규 아키텍처로 완전히 전환했습니다.

4.1 3단계 분리 파이프라인 정립

단계	실행 위치 및 함수	역할 및 상세 내용
Phase 1: 입력 모달 (Local Client)	`UMageSearchLibraryTacticalSkill::InitiateSkill` → `PC->RequestCardSelection(Context)`	하드코딩된 컨트롤러 래퍼를 거치지 않고 범용 `FCardSelectionContext`를 구성하여 로컬 플레이어에게 카드 선택 모달(창)을 띄움.
Phase 2: 서버 검증/실행 (Dedicated Server)	`PC->Server_SelectCardForLibrary` → `USearchLibraryEffect::ApplyEffect`	선택된 카드의 유효성을 서버에서 검증하고, 덱 최상단 배치 및 카드 드로우 로직 수행.
Phase 3: 시각 연출 통지 (NetMulticast)	`SearchLibraryEffect.cpp` → `BroadcastVisualCue(CueData)`	`Visual.TacticalSkill.Mage.SearchLibrary` 태그와 함께 시전자/타깃 정보를 담아 전체 클라이언트로 브로드캐스트.
Phase 4: 독립 연출 재생 (Client HUD)	`UWildHUDWidget::HandleVisualCue` → `WBP_SearchLibraryEffect` 스폰	메인 HUD 수정 없이, 등록된 `WBP_SearchLibraryEffect` 위젯이 생성되어 카드 드로우 및 셔플 연출을 재생한 후 자동 소멸.

4.2 리팩터링 전후 코드 비교

[Before] PlayerUnit의 하드코딩된 오염 코드:

```
// [기존 PlayerUnit.cpp] - 모든 전술스킬이 동일한 이펙트/사운드만 재생 가능했음
void APlayerUnit::ExecuteTacticalSkillAction(const FVector& TargetLocation)
{
    // ...
    if (AbilityVFX) UGameplayStatics::SpawnEmitterAtLocation(...);
    if (AbilitySound) UGameplayStatics::PlaySoundAtLocation(...);
    OnHeroAbilityTriggered(); // 단일 이벤트로 전술스킬별 개별 UI 분기 불가!
}
```

[After] 리팩터링 후 완전 분리된 구조:

```
// [개선 후 PlayerUnit.cpp] - 오염 코드 완전 제거, 순수 상태 변경만 수행
void APlayerUnit::ExecuteTacticalSkillAction(const FVector& TargetLocation)
{
    if (CurrentTacticalSkill)
    {
        CurrentTacticalSkill->ExecuteSkill(this, TargetLocation);
    }
}

// [개선 후 SearchLibraryEffect.cpp] - Effect가 필요한 시점에 정밀한 Visual Cue 발송
bool USearchLibraryEffect::ApplyEffect(const FEffectContext& Context)
{
    // 덱 조작 및 드로우 비즈니스 로직 수행 ...

    // 시각적 연출 브로드캐스트
    FWildVisualCueData CueData;
    CueData.CueTag = FGameplayTag::RequestGameplayTag(TEXT("Visual.TacticalSkill.Mage.SearchLibrary"));
    CueData.Instigator = Context.SourceUnit;
    CueData.Target = Context.TargetUnit;
    CueData.Value = TopCards.Num();
    BroadcastVisualCue(Context, CueData);
    return true;
}
```

5. Git 협업 생산성 및 아키텍처 확장성 비교 분석

5.1 Before vs After 비교 분석표

비교 항목	기존 (Legacy Architecture)	개선 후 (Unified Event Bus Architecture)
WBP_HUD 의존도	모든 스킬/이펙트 연출이 WBP_HUD 내부에 직접 구현됨	**Zero-Touch** (WBP_HUD는 라우터 역할만 수행, 수정 불필요)
Git 바이너리 충돌	이펙트 작업자 간 `WBP_HUD.uasset` 충돌 빈번	**충돌 0%** (1개 연출 = 1개 독립 위젯 에셋 분리)
C++ 코드 결합도	PlayerUnit, PlayerController에 개별 연출/UI 래퍼 함수 난립	`BroadcastVisualCue` 및 `VisualCueTag` 단일 인터페이스로 통일
신규 연출 추가 비용	C++ 컴파일 → PlayerUnit 수정 → WBP_HUD 수정 (3중 수정)	데이터 테이블 태그 등록 → 독립 WBP 제작 (C++ 및 HUD 수정 없음)
네트워크 오버헤드	스킬/이펙트별로 별도의 Multicast RPC를 지속 선언	단일 `Multicast_BroadcastVisualCue` RPC로 공유

5.2 다중 작업자 동시 작업 파이프라인 (Git-Safe Workflow)

개선된 아키텍처에서는 새로운 카드 효과나 전술스킬이 기획되었을 때, 작업자 간에 단 하나의 파일도 겹치지 않고 완전 병렬로 개발을 진행할 수 있습니다.

작업자별 역할 분담 및 변경 파일 독립성:

- **기획자 (Designer):**
  - GameplayTag 신규 등록 (예: `DefaultGameplayTags.ini` 또는 프로젝트 세팅)
  - `DT_CardData` 또는 `DT_TacticalSkillData` 데이터 테이블에 `VisualCueTag` 지정 (텍스트/CSV 파일 기반 충돌 없음)
- **서버/게임플레이 프로그래머 (Programmer):**
  - `UEffectBase`를 상속받는 신규 C++ 이펙트 클래스 작성 (수치 로직 처리 후 `BroadcastVisualCue` 호출)
  - HUD나 위젯 코드를 건드릴 필요가 전혀 없음
- **UI / 테크니컬 아티스트 (Tech Artist):**
  - `UWildVisualEffectWidget`을 상속받는 개별 위젯 블루프린트 생성 (예: `WBP_FireballEffect.uasset`)
  - UMG 애니메이션, 파티클, 사운드 연출 독립 제작
  - 프로젝트 설정 또는 데이터 테이블에서 태그 매핑 등록
- **공통 메인 UI (WBP_HUD):**
  - **완전 무변경 (Zero Touch).** 바이너리 충돌 가능성 원천 차단.

6. 소스 코드 리팩터링 상세 내역 및 빌드 검증

6.1 수정 및 신규 생성된 C++ 파일 목록 (총 10종)

파일 경로	구분	주요 변경 및 구현 내용
`InGame/Data/BattleData.h`	수정	`FWildVisualCueData` 구조체 정의, `FCardData` / `FTacticalSkillData` / `FEffectData`에 `VisualCueTag` 추가
`InGame/Game/InGameState.h/.cpp`	수정	`Multicast_BroadcastVisualCue` RPC 및 `OnVisualCueReceivedDelegate` 구현
`InGame/Effect/EffectBase.h/.cpp`	수정	모든 자식 이펙트에서 공용으로 사용할 수 있는 `BroadcastVisualCue` 헬퍼 메서드 추가
`InGame/Effect/DamageEffect.cpp`	수정	데미지 적용 후 `Visual.Effect.Damage` 태그 및 데미지 수치 Visual Cue 발송 연동
`InGame/Effect/HealEffect.cpp`	수정	체력 회복 후 `Visual.Effect.Heal` 태그 및 회복량 Visual Cue 발송 연동
`InGame/Effect/SearchLibraryEffect.cpp`	수정	서고 카드 드로우 완료 후 `Visual.TacticalSkill.Mage.SearchLibrary` 발송 연동
`InGame/Battle/ChainComponent.cpp`	수정	`CastSpell` 시 데이터 테이블의 `VisualCueTag` 브로드캐스트 연동
`InGame/UI/WildVisualEffectWidget.h/.cpp`	신규	독립 연출 위젯의 공통 베이스 클래스 구현 (Play/Finish 라이프사이클)

파일 경로	구분	주요 변경 및 구현 내용
`InGame/UI/WildHUDWidget.h/.cpp`	수정	`VisualEffectWidgetMap` 정의, 계층형 태그 매칭 및 동적 스폰/풀링 라우터 구축
`InGame/Unit/PlayerUnit.cpp`	수정	`ExecuteTacticalSkillAction` 내 하드코딩된 단일 VFX/SFX 제거 및 결합도 해소
`InGame/Unit/TacticalSkillBase.h/.cpp`	수정	`VisualCueTag` 프로퍼티 추가, `UMageSearchLibraryTacticalSkill`을 범용 `RequestCardSelection` 구조로 리팩터링

6.2 빌드 검증 결과

언리얼 엔진 5.7 빌드 결과 (UnrealBuildTool):

- **빌드 타깃:** `WildCardEditor Win64 Development`
- **결과: Result: Succeeded (Exit Code 0)**
- **검증 상태:** 문법 오류, 링크 에러, 리플렉션(UHT) 충돌 없이 완벽하게 컴파일 완료.

7. 결론 및 향후 권장 작업

본 리팩터링을 통해 WildCard 프로젝트는 카드 효과와 전술스킬의 시각 연출을 완전히 데이터 주도형(Data-Driven)으로 분리하였으며, Git 머지 충돌 없는 안전한 협업 기반을 마련했습니다.

- **블루프린트 에셋 매핑:** 에디터 실행 후 `WBP_HUD`의 클래스 디폴트에서 `VisualEffectWidgetMap`에 GameplayTag별 위젯 클래스 매핑.
- **서고 탐색 연출 위젯 제작:** `UWildVisualEffectWidget`을 상속받는 `WBP_SearchLibraryEffect`를 제작하여 멋진 카드 공개 및 드로우 애니메이션 적용.